



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

OlympusX

Follow-Up Review

May 2026

Prepared by

ZanyBonzy

Varun_Sharma

radev_eth

Introduction

A time-boxed security review of the **OlympusX** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A code review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About OlympusX

OlympusX is a DeFi yield protocol on Ethereum built around a deflationary ERC20 token (OLY) with an 8B fixed supply. The protocol launches through a 45-day mint phase with three vesting schedules, dynamic daily pricing (2% increment), and a referral bonus system. Post-launch, it generates yield through three channels: Lido ETH staking, Uniswap V3/V4 liquidity farming, and automated buy-and-burn. All protocol revenue flows through TaxRouter, which splits incoming ETH across five destinations - stakers, buy-and-burn, validators, farms, and genesis - at configurable BPS ratios. Trading operates through a custom Uniswap V4 hook that enforces dynamic sell taxes across four market-cap tiers (8%-38%), manages on-chain limit orders with epoch-based fill tracking, and captures all LP swap fees for the protocol.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

review commit hash - [ed91d631cb5ceb244d2e73e3a38fbbbeb078fb6ea](#)

This report covers a separate follow-up audit focused only on the newly introduced code changes after the initial audit, together with the additional **BuyAndBurn.sol** contract added to scope.

Scope

The following smart contracts changes were in scope of the audit:

- <https://github.com/De-centraX/olympusx-contracts/pull/99>
- <https://github.com/De-centraX/olympusx-contracts/pull/98>
- <https://github.com/De-centraX/olympusx-contracts/pull/97>
- <https://github.com/De-centraX/olympusx-contracts/pull/71>
- `./src/BuyAndBurn.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 1 issues
- Medium: 2 issues
- Low and Informational: 7 issues

Findings Summary

ID	Title	Severity	Status
[H-01]	Reentrancy still present in <code>buyNburn()</code>	High	Fixed
[M-01]	BuyAndBurn pays the caller incentive even on failed swaps	Medium	Fixed
[M-02]	Transfer tax is burned instead of routed when no tax receiver is set	Medium	Acknowledged
[L-01]	BuyAndBurn missing <code>_disableInitializers</code> and deployed without proxy	Low	Acknowledged
[L-02]	Reward tokens can never be removed, so one broken token bricks batch claims for everyone	Low	Acknowledged
[L-03]	BuyAndBurn never bounds the incentive percentage and has no way to fix a bad value	Low	Fixed
[L-04]	<code>totalOlympusXBurnt</code> is updated even when some other token is burnt	Low	Fixed
[I-01]	<code>calculateCurrentIncentivePercentage</code> returns incorrect value when time elapsed is < <code>intervalBetweenBurns</code>	Informational	Fixed
[I-02]	Cast <code>intervalBetweenBurns</code> to uint256 before setting <code>incentiveRampDuration</code>	Informational	Fixed
[I-03]	<code>proposalThreshold</code> denominator is BPS (10 000) while <code>quorumDenominator</code> is percent (100)	Informational	Fixed

Detailed Findings

[H-01] Reentrancy still present in `buyNBurn()`

Severity

Impact:

High

Likelihood:

High

Description

Following is `buyNBurn` function:

```
function buyNBurn(uint48 _deadline) external {
    .....
    .....

    ===?    _safeETHTransfer(msg.sender, incentive);

            emit BuyAndBurnExecuted(result.amountOut);

    ===>    lastBurnTs = uint32(block.timestamp);
}
```

In the above function still the `lastBurnTs` is updated after the ETH transfer call due to which reentrancy bug is still present.

Recommendations

Update the `lastBurnTs` before transferring the ETH.

[M-01] BuyAndBurn pays the caller incentive even on failed swaps

Severity

Impact:

High

Likelihood:

Low

Description

`SwapActions._swapV4Currency` is designed to swallow `SwapActionsLib__TooLittleReceived` error and other reverts and shrink the amount on each retry; if every retry trips slippage the function returns successfully with `result.amountOut == 0` and `retryData.isSuccess == false` due to being wrapped in a try-catch mechanism.

```
        while (retryData.amountToSwap > 0 && retryData.retryCount <
maxRetries) {
            try SwapActionsLib.swapExactInV4(
                IUniversalRouter(v4Router),
                poolKey,
                currencyIn,
                currencyOut,
                uint128(retryData.amountToSwap),
                result.twapSqrtPriceX96,
                params.slippage
            ) returns (
                uint256 amountOut
            ) {
                result.amountOut = amountOut;
                retryData.isSuccess = true;
                break;
            } catch (bytes memory reason) {
                retryData.amountToSwap =
                    uint128(_getNewSwapAmountForRetry(reason,
retryData.amountToSwap, swapAmountForRetryBPS));
            }
            unchecked {
                ++retryData.retryCount;
            }
        }
    }
```

`BuyAndBurn.buyNBurn` never checks the result and just proceeds:

```
        SwapActions.SwapExactInResult memory result =
_swapV4Currency(...);
        burn(); // burns 0 OLY
        _safeETHTransfer(msg.sender, incentive); // still pays
the caller
        emit BuyAndBurnExecuted(result.amountOut); // emits 0
        lastBurnTs = uint32(block.timestamp);
```

So a caller can intentionally trigger `buyNBurn` exactly when the TWAP-vs-spot deviation guarantees that every retry will revert with `TooLittleReceived` (e.g. immediately after a large sell, or as part of a self-

engineered short squeeze), collect the incentive, and contribute nothing back to the protocol. They also push `lastBurnTs` to `block.timestamp`, blocking honest callers from the next interval window.

Combined with the reentrancy issue already mentioned, a malicious user can drain the contract of ETH via incentives.

Recommendations

After the swap, require a positive output before paying the incentive and updating `lastBurnTs`.

[M-02] Transfer tax is burned instead of routed when no tax receiver is set

Severity

Impact:

Medium

Likelihood:

Medium

Description

`OlympusX.sol#_update()` taxes transfers that involve a taxable address and is supposed to send the tax to a receiver, which then feeds `OlyTaxSwapper` -> `TaxRouter` -> stakers, buy-and-burn, validators, farms and genesis.

The receiver is resolved from per-address overrides:

```
address receiver = transferTaxReceiverOverride[from];
if (receiver == address(0)) {
    receiver = transferTaxReceiverOverride[to];
}
super._update(from, receiver, taxAmount);
```

If neither side has an override set, `receiver` stays `address(0)` and `super._update(from, address(0), taxAmount)` is a burn - the tax is destroyed and never reaches any recipient.

The problem is that turning the tax on and setting where it goes are two separate steps that nothing ties together. `setTransferTaxWhitelist` activates the tax on a pool;

`setTransferTaxReceiverOverride` sets the destination. There's no check in `_update` requiring a receiver when tax is charged, no revert and no distinct event - so the tax just leaks silently.

This is the default state after deployment. The scripts wire up `OlyTaxSwapper` and whitelist it as a tax setter, but never call `setTransferTaxReceiverOverride`. The moment someone marks a pool taxable (the natural way to switch the system on), every taxed transfer burns the tax until the receiver override is

also set in a separate call that's easy to forget. The `taxBps == 0` early return doesn't help, since any working tax config has a non-zero rate.

Each taxed transfer during that window is an irreversible loss of protocol revenue (not principal), which is why this is Medium.

Recommendations

Require a non-zero receiver whenever tax is actually charged, so a missing override reverts the transfer instead of burning the tax:

```
address receiver = transferTaxReceiverOverride[from];
if (receiver == address(0)) {
    receiver = transferTaxReceiverOverride[to];
}
require(receiver != address(0), OlympusX__TransferTaxReceiverNotSet());

super._update(from, receiver, taxAmount);
super._update(from, to, netAmount);
```

[L-01] BuyAndBurn missing `_disableInitializers` and deployed without proxy

Description

PR97 converted all protocol contracts to the initializer pattern. Every upgradeable contract - StakingVault, FarmKeeper, TaxRouter, OlyTaxSwapper, OlympusXAuction, LaunchManager, EthPrice calls `_disableInitializers()` in its constructor and is deployed atomically behind an `ERC1967Proxy` via the `_proxy()` helper.

`BuyAndBurn` is the sole exception on both counts. It has no constructor, no `_disableInitializers()` and the deploy script performs a bare deploy followed by a separate `initialize()` call:

```
buyAndBurn = new BuyAndBurn();
buyAndBurn.initialize(
    msg.sender,
    address(olympusX),
    address(universalRouter),
    ...
);
```

`initialize()` has no caller restriction. only the `initializer` modifier prevents a second call. Because `new BuyAndBurn()` and `buyAndBurn.initialize()` are separate transactions within `vm.startBroadcast()`, there is a window where a front-runner can call `initialize()` with their own address and become the contract owner.

In practice, this would cause the deployer `initialize()` to revert, halting the Foundry broadcast before the downstream wiring calls (`TaxRouter.setBuyAndBurn`, `ValidatorVault.setBuyAndBurn`, `FarmKeeper.setBuyAndBurn`) ever execute.

This is basically inconsistency with every other contract in the protocol.

Recommendations

Add a constructor with `_disableInitializers()` to match every other initializable contract and deploy through the existing `_proxy()` helper for atomic initialization.

[L-02] Reward tokens can never be removed, so one broken token bricks batch claims for everyone

Description

`StakingVault.sol#addRewardToken()` registers a token and flips `isRewardToken[_token]` to true, but there is no function anywhere to undo that. Registration is permanent.

The batch withdrawal path loops over every registered token and calls `safeTransfer` on each (in `StakingRewardsLib.withdrawTokens()`). If any one of those tokens ever reverts on transfer, a pausable token that gets paused, a token whose blacklist catches the vault, an upgradeable token that breaks, then `claimAllRewardsAndWithdraw()` and `withdrawBatch()` revert for every user and there is no way to deregister the offending token to unblock them.

Also the code comment in `StakingVault.sol#_claimRewardForToken()`: "Users can claim rewards even for removed tokens." The design clearly assumed tokens could be removed and the removal function just never got written. The per-token claim paths still work as a fallback, but a single bad reward token shouldn't be able to brick the batch path permanently.

Recommendations

Add an owner/governance `removeRewardToken(address)` that swap-pops the token out of `rewardTokens` and clears `isRewardToken[token]`, leaving `cumulativeRewardPerShare` and `pendingWithdrawals` untouched so already-accrued rewards stay claimable through the single-token path.

[L-03] BuyAndBurn never bounds the incentive percentage and has no way to fix a bad value

Description

In `BuyAndBurn.sol#buyNBurn()`, the swap amount is `balance - incentive`, where `incentive = balance * incentivePct / 1e18`. If `maxIncentivePercentage` is ever at or above 100% (1e18), then once the incentive ramp hits max the incentive meets or exceeds the balance, the subtraction underflows and `buyNBurn` reverts every time, basically buy-and-burn is dead.

The problem is that `baseIncentivePercentage` and `maxIncentivePercentage` are set only in `initialize()`, which does no bounds check, and there is no setter to change them afterward. So a bad value at deployment permanently bricks buy-and-burn with no on-chain recovery short of redeploying and re-pointing `TaxRouter.setBuyAndBurn()`.

The sibling contract already guards this exact case: `OnlyTaxSwapper.setIncentiveParams()` requires `maxIncentivePercentage < PRECISION` and `maxIncentivePercentage >= baseIncentivePercentage`. `BuyAndBurn` is just missing both the check and the setter.

Recommendations

Add an `onlyOwner setIncentiveParams(uint256 base, uint256 max)` that enforces `max < PRECISION` and `max >= base` and apply the same two checks inside `initialize()`. That both prevents the misconfiguration and makes it recoverable.

[L-04] `totalOlympusXBurnt` is updated even when some other token is burnt

Description

Following `burn()`:

```
function burn(address _token) public {
    uint256 toBurn = ERC20Burnable(_token).balanceOf(address(this));
    if (toBurn > 0 && _token == address(olympusX)) {
        IStakingVault(launchManager.getStakingVault()).syncInflation();
    }
    totalOlympusXBurnt += toBurn;

    ERC20Burnable(_token).burn(toBurn);
    emit TokensBurned(_token, toBurn);
}
```

Now we can clearly see that the `totalOlympusXBurnt` is updated outside the if statement due to this if some other token is burnt still the total olympus burnt variable is updated which is incorrect.

Recommendations

Keep the update of `totalOlympusXBurnt` inside the if statement.

[I-01] `calculateCurrentIncentivePercentage` returns incorrect value when time elapsed is < `intervalBetweenBurns`

Description

When the time elapsed between the last buyNburn call is less than interval between burns then in this case no buyNburn call happens and hence there is zero incentive given to the call but the calculateCurrentIncentive percentage function is a public function which returns baseIncentivePercentage which is incorrect.

```
function calculateCurrentIncentivePercentage() public view returns
(uint256 incentivePercentage) {
    uint32 timeElapsed = uint32(block.timestamp) - lastBurnTs;

    // First buyNBurn is always with base incentive
    // Ensure ramping starts only after the interval between burns has
passed
    ==>    if (lastBurnTs == 0 || timeElapsed < intervalBetweenBurns) {
        return baseIncentivePercentage;
    }

    uint32 rampingTimeElapsed = timeElapsed - intervalBetweenBurns;

    if (rampingTimeElapsed >= incentiveRampDuration) {
        return maxIncentivePercentage;
    }

    // Linear interpolation between base and max incentive
    uint256 additionalIncentive =
        (maxIncentivePercentage -
baseIncentivePercentage).mulDiv(rampingTimeElapsed, incentiveRampDuration);

    incentivePercentage = baseIncentivePercentage +
additionalIncentive;
}
```

It correctly returns base incentive percentage when last burns ts is zero but when time elapsed is less than interval between burns then it should return zero instead of base incentive percentage. Similar logic applies to the olySwapper contract where this incentive logic is used as well.

Recommendations

In case of time elapsed less than interval between burns/ interval between swaps return 0.

[I-02] Cast `intervalBetweenBurns` to `uint256` before setting `incentiveRampDuration`

Description

In `BuyAndBurn.setConfig`:

```
incentiveRampDuration = intervalBetweenBurns * 2;
```

`intervalBetweenBurns` is `uint32`. Solidity 0.8 evaluates the right-hand side as a `uint32` expression, then widens to `uint256` for the assignment — so the multiplication itself overflow-checks at `uint32`.

By contrast `OlyTaxSwapper.setConfig` cleanly widens first:

```
incentiveRampDuration = uint256(_intervalBetweenSwaps) * 2;
```

Recommendations

Cast before multiplying to match `OlyTaxSwapper`:

```
incentiveRampDuration = uint256(intervalBetweenBurns) * 2;
```

[I-03] `proposalThreshold` denominator is BPS (10 000) while `quorumDenominator` is percent (100)

Description

`OlympusXGovernor` mixes two different fractional bases for two parameters that look identical at the call site:

- `PROPOSAL_THRESHOLD_DENOMINATOR = 10_000` (basis points) — used for `_proposalThresholdNumerator`. The constructor's natspec says "E.g. 50 == 0.5%".
- `GovernorVotesQuorumFraction.quorumDenominator()` returns `100` by default — used for `_quorumNumerator`. Here `4` means 4%.

Both parameters arrive in the constructor as `uint256` with similar names (`_proposalThresholdNumerator`, `_quorumNumerator`) and no on-chain bound that distinguishes them. A deployer who reads "50 = 0.5%" and copies the convention to `_quorumNumerator = 50` ends up with a **50% quorum** instead of 0.5%, instantly soft-bricking governance (and conversely, treating quorum-style "4" as 4% on the threshold yields 0.04% threshold — proposal spam).

Recommendations

Align both denominators by overriding `quorumDenominator()` to return `10_000` (BPS).